Quantum Computing and Quantum Machine Learning

Morten Hjorth-Jensen[1]

Department of Physics, University of Oslo[1]

April 9

Plans for the week of April 7-11, 2025

1. Reminder from last week about Quantum Fourier Transforms
2. Quantum phase estimation algorithm, final derivation and discussions
3. Brief discussion of Shor's algorithm
4. Reading recommendation: Hundt section 6.4 for the QPE and section 6.5 for Shor's algorithm

Reminder on the Quantum Fourier transform

The Quantum Fourier Transform (QFT) is a key component in many quantum algorithms, notably Shor's factoring algorithm and quantum phase estimation. It serves as the quantum analogue of the classical Discrete Fourier Transform (DFT), but operates exponentially faster due to the principles of quantum parallelism and entanglement.

Classical Discrete Fourier Transform

Given a vector of N complex numbers $\{x_0, x_1, \ldots, x_{N-1}\}$, the DFT is defined as:

$$X_k = \sum_{n=0}^{N-1} x_n \exp -(2\pi ikn/N), \quad k = 0, 1, \ldots, N-1$$

The inverse transform is:

$$x_n = \frac{1}{N} \sum_{k=0}^{N-1} X_k \exp\left(2\pi ikn/N\right)$$

In matrix form

This can be expressed in matrix form:

$$\boldsymbol{X} = F_N \boldsymbol{x}, \quad \text{where} \quad (F_N)_{jk} = \frac{1}{\sqrt{N}} \exp\left(2\pi i j k / N\right)$$

Quantum Fourier Transform

Let $N = 2^n$. The QFT acts on the state:

$$|x\rangle = |x_0 x_1 \dots x_{n-1}\rangle$$

and transforms it into:

$$\mathrm{QFT}|x\rangle = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} \exp\left(2\pi i x k / N\right)|k\rangle$$

The QFT is unitary and can be decomposed into a series of Hadamard and controlled phase gates, allowing for an efficient $O(n^2)$ implementation.

Comparison with Classical DFT

While the classical FFT runs in $O(N \log N)$ time, the QFT runs in $O(n^2) = O(\log^2 N)$ gate operations—an exponential improvement, crucial in quantum speedups for specific problems.

Unitary Nature of QFT

Last week we showed that the QFT matrix F_N is unitary:

$$F_N^\dagger F_N = I$$

This guarantees that QFT is a valid quantum operation (i.e., norm-preserving and reversible).

PennyLane Setup and Basic Example

To experiment with QFT, we can use the PennyLane Python library.

```
import pennylane as qml
from pennylane import numpy as np

n_qubits = 3
dev = qml.device("default.qubit", wires=n_qubits)

def apply_qft(wires):
    qml.templates.QFT(wires=wires)

@qml.qnode(dev)
def qft_circuit():
    qml.BasisState(np.array([1, 0, 1]), wires=range(n_qubits))
    apply_qft(wires=range(n_qubits))
    return qml.state()

state = qft_circuit()
print("QFT output state:", state)
```

Quantum Circuit Implementation

The Quantum Fourier Transform can be efficiently implemented using a combination of Hadamard gates and controlled phase gates. Given an n-qubit state $|x\rangle$, the QFT circuit transforms it into the state:

$$\frac{1}{\sqrt{2^n}} \sum_{k=0}^{2^n-1} \exp\left(2\pi i x k / 2^n\right) |k\rangle$$

This is achieved through a decomposition into a quantum circuit of $O(n^2)$ gates.

QFT on three qubits

We illustrate the circuit for $n = 3$. The transformation proceeds as follows:

1. Apply a Hadamard gate to the first qubit.
2. Apply controlled-R_k gates (phase gates) for $k = 2, 3, \ldots$ to encode phase interference.
3. Repeat recursively on the remaining qubits.
4. Apply a qubit swap at the end to reverse the order of qubits.

Controlled Phase Rotation

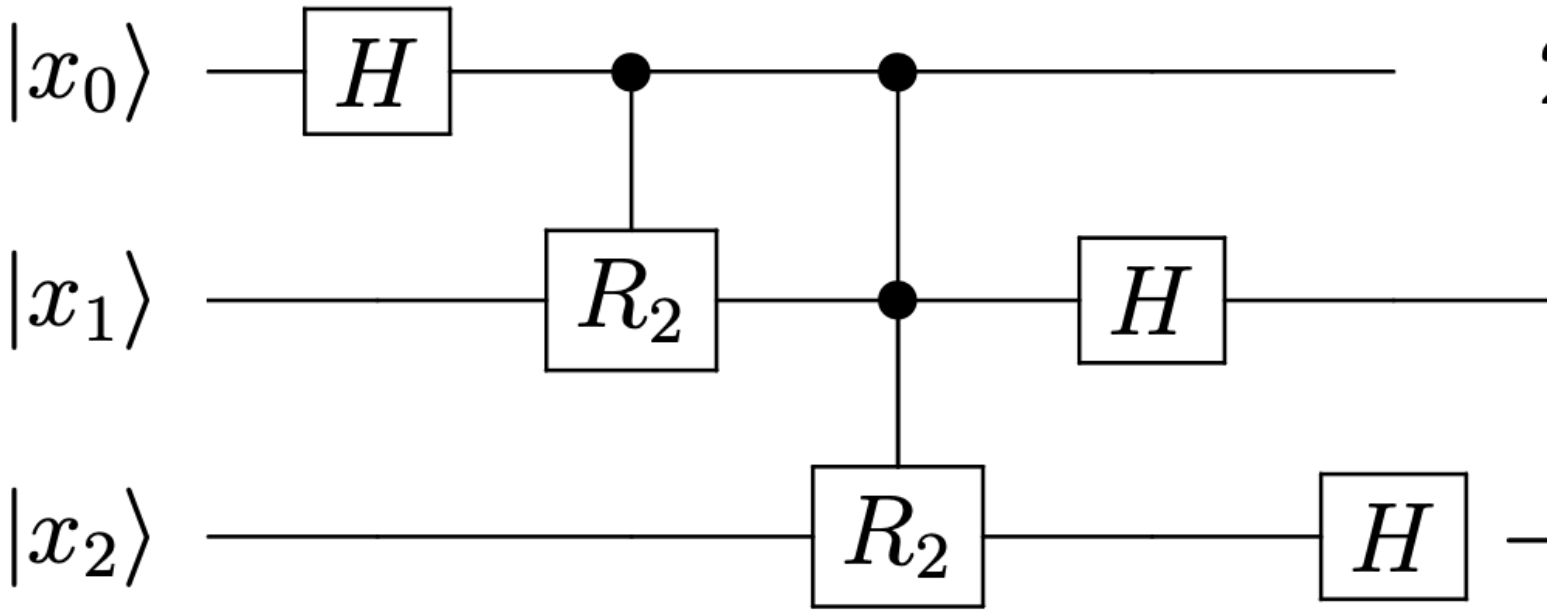
The controlled phase rotation gate R_k is defined as:

$$R_k = \begin{bmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^k} \end{bmatrix}$$

This gate introduces the correct phase into the wavefunction based on the binary digits of the input number.

Circuit Diagram

The QFT circuit for three qubits is shown below (read top to bottom):



Python code with PennyLane

We now implement a full QFT circuit manually for three qubits using PennyLane.

```
import pennylane as qml
from pennylane import numpy as np
import math

dev = qml.device("default.qubit", wires=3)

def qft_rotations(wires):
    if len(wires) == 0:
        return
    head, *tail = wires
    qml.Hadamard(wires=head)
    for i, qubit in enumerate(tail):
        qml.ctrl(qml.PhaseShift, control=qubit)(math.pi / 2**(i+1), wir
    qft_rotations(tail)

def swap_registers(wires):
    for i in range(len(wires) // 2):
        qml.SWAP(wires=[wires[i], wires[-i - 1]])

@qml.qnode(dev)
def qft_custom():
    qml.BasisState(np.array([1, 0, 1]), wires=range(3))
    qft_rotations(wires=[0, 1, 2])
    swap_registers(wires=[0, 1, 2])
    return qml.state()
```

Complexity

1. The QFT requires $O(n^2)$ gates.
2. Approximate QFT implementations can reduce complexity by omitting small-angle phase gates.

The circuit implementation of QFT is surprisingly efficient, enabling its use in several key quantum algorithms. Understanding the gate-level construction deepens our understanding of quantum information flow during Fourier transformation.

Inverse QFT and the phase estimation algorithm

In our discussion of the phase estimation algorithm, we will need the inverse QFT. The inverse Quantum Fourier Transform (IQFT) is simply the Hermitian adjoint of the QFT. Since the QFT is unitary, its inverse is:

$$\mathrm{QFT}^{\dagger} = \mathrm{QFT}^{-1}$$

Circuit Construction

To reverse the QFT circuit:

1. Swap the order of gates.
2. Replace Hadamard gates with Hadamard gates (self-inverse).
3. Replace each R_k gate with its conjugate transpose $R_k^\dagger$:

$$R_k^\dagger = \begin{bmatrix} 1 & 0 \\ 0 & e^{-2\pi i/2^k} \end{bmatrix}$$

Codes for the QFT

Here we present examples on how we can build our own QFT codes, without using software like PennyLane or QisKit. The first code is a simple code without the introduction of a class structure. Its key features are

1. Inverse QFT (via negative controlled-phase angles and reversed order),
2. Measurement sampling (random draws from the final state's probability distribution).
3. Forward and inverse QFT logic built in.
4. Uses exact Hadamard and controlled phase gates.
5. Simulates quantum measurement from the QFT result.
6. Bit-reversal swaps ensure correct output basis alignment.

```
import numpy as np

def hadamard():
    return np.array([[1, 1], [1, -1]]) / np.sqrt(2)

def apply_single_qubit_gate(state, gate, target, n_qubits):
    """Applies a single-qubit gate to the target qubit."""
    I = np.eye(2)
    ops = [I] * n_qubits
    ops[target] = gate
```

Adding classes

This variant of the same code attempts at making a more structured introduction to classes. It introduces a reusable Python class called QuantumFourierTransform that encapsulates everything: It includes the same elements as before but adds also a visualization of the measurements. Here we have

1. Initialization of state
2. QFT and inverse QFT
3. Measurement sampling
4. Easy customization of qubit number and input state
5. Includes state visualization and measurement simulation.

The benefits of the class design is that the code is reusable and hopefully clean and makes it easy to switch between QFT and inverse QFT.

```
import numpy as np
class QuantumFourierTransform:
    def __init__(self, n_qubits):
        self.n_qubits = n_qubits
        self.N = 2 ** n_qubits
        self.state = np.zeros(self.N, dtype=complex)

    def initialize_basis_state(self, index):
```

Plotting the amplitudes

```
import numpy as np
import matplotlib.pyplot as plt
class QuantumFourierTransform:
    def __init__(self, n_qubits):
        self.n_qubits = n_qubits
        self.N = 2 ** n_qubits
        self.state = np.zeros(self.N, dtype=complex)

    def initialize_basis_state(self, index):
        """Set state to basis state |index⟩"""
        self.state = np.zeros(self.N, dtype=complex)
        self.state[index] = 1.0

    def initialize_custom_state(self, state_vector):
        """Set state to custom normalized state vector"""
        assert len(state_vector) == self.N, "State vector length misma
        norm = np.linalg.norm(state_vector)
        assert np.isclose(norm, 1.0), "State vector must be normalized
        self.state = np.array(state_vector, dtype=complex)

    def initialize_superposition(self):
        """Create |+⟩^n superposition state"""
        self.state = np.ones(self.N, dtype=complex) / np.sqrt(self.N)
    def initialize_bell_pair(self):
        """2-qubit Bell state: (|00⟩ + |11⟩)/2"""
        if self.n_qubits != 2:
            raise ValueError("Bell pair requires exactly 2 qubits.")
        self.state = np.zeros(self.N, dtype=complex)
```

Adding animations

This code example adds superposition inputs to the previous code (e.g. Hadamards to create $|+\rangle^{\otimes n}$) and customs a state initialization (e.g. any normalized complex vector). It includes also the possibility of having entangled state preparation (e.g. Bell states, GHZ states, etc.) It has the following added feature

1. initialize_superposition() gives $|+\rangle^{\otimes n}$
2. initialize_bell_pair() for two-qubit entangled states
3. initialize_ghz_state() creates a GHZ state
4. initialize_custom_state(vec) allows to pass any normalized state

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
import os

class QuantumFourierTransform:
    def __init__(self, n_qubits):
        self.n_qubits = n_qubits
        self.N = 2 ** n_qubits
        self.state = np.zeros(self.N, dtype=complex)

    def initialize_basis_state(self, index):
        """Set state to basis state |index⟩"""
```

Introduction to Quantum Phase Estimation (QPE)

The QPE is a fundamental quantum algorithm used to estimate the phase ϕ in eigenvalue equations of unitary operators. Given a unitary operator U and an eigenvector $|\psi\rangle$ such that:

$$U|\psi\rangle = \exp(2\pi i\phi)|\psi\rangle$$

the goal of QPE is to estimate ϕ.
The QPE provides an estimate of ϕ with high probability. It is a crucial subroutine for algorithms like

1. Shor's factoring algorithm and
2. quantum many-body simulations.

Algorithm overview

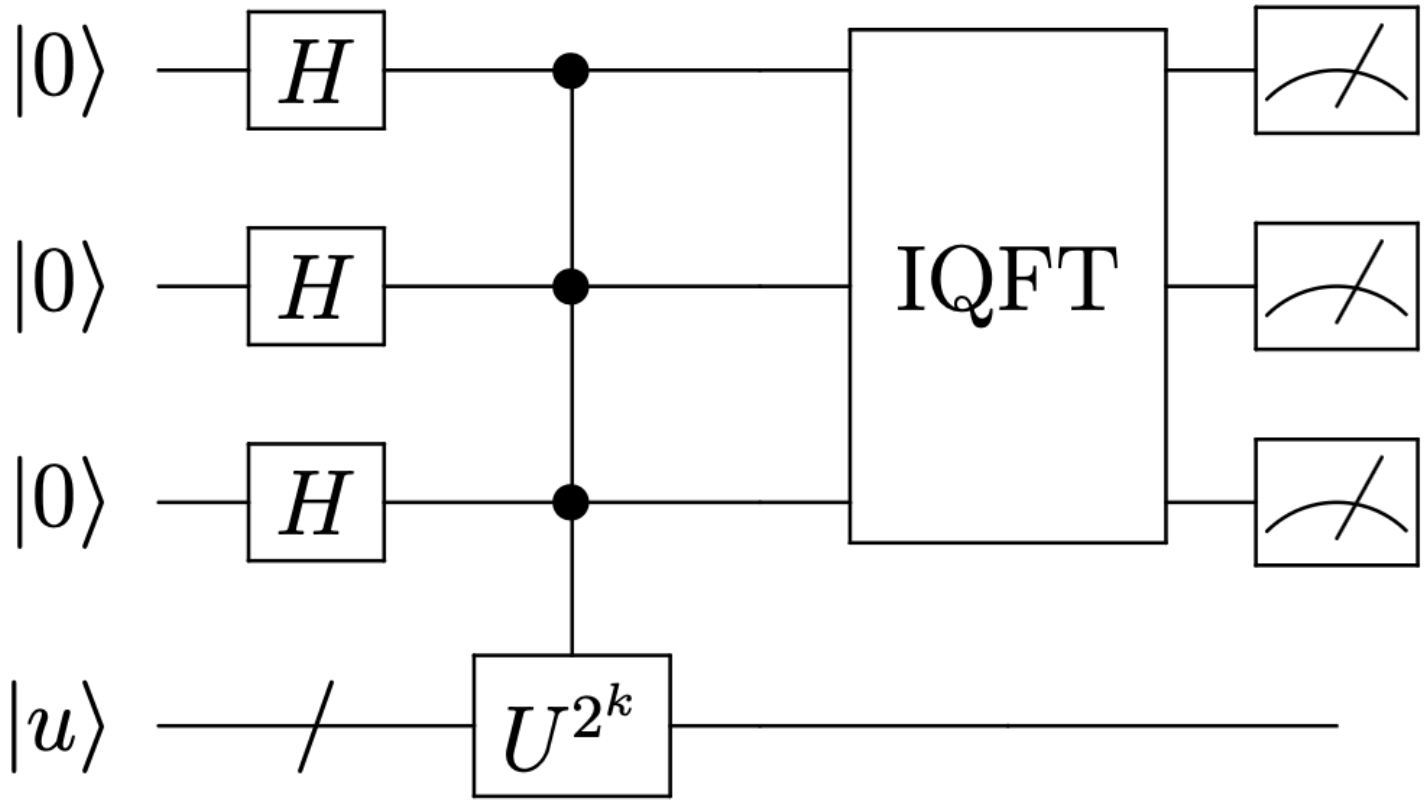
Given an eigenvector $|u\rangle$ of a unitary U, the QPE algorithm estimates the phase ϕ using:

1. Apply Hadamards to an n-qubit register.
2. Use controlled-U^{2^k} gates to encode phase.
3. Apply inverse QFT to extract the binary digits of ϕ.

Circuit diagram (three-qubit QPE)

The quantum circuit for QPE consists of two quantum registers:

1. The first register with n qubits is initialized to $|0\rangle^{\otimes n}$.
2. second register is initialized to the eigenstate $|\psi\rangle$ of U.



Mathematical Background

The goal of QPE is to estimate the phase $\phi \in [0, 1)$ where:

$$U|\psi\rangle = \exp(2\pi i\phi)|\psi\rangle.$$

The Quantum Fourier Transform on n qubits is defined as (see above):

$$|x\rangle \to \frac{1}{\sqrt{2^n}} \sum_{y=0}^{2^n-1} e^{2\pi ixy/2^n}|y\rangle.$$

The QFT is essential for extracting phase information in QPE.

Derivation of the Algorithm, Step 1: Initialization

The system starts in the state:

$$|0\rangle^{\otimes n} \otimes |\psi\rangle.$$

Applying Hadamard gates to the first register creates a superposition:

$$\frac{1}{2^{n/2}} \sum_{k=0}^{2^n-1} |k\rangle \otimes |\psi\rangle.$$

Step 2: Controlled Unitary Operations

Controlled-U^{2^j} gates apply the operation U with exponential powers:

$$\frac{1}{2^{n/2}} \sum_{k=0}^{2^n-1} |k\rangle \otimes U^k |\psi\rangle.$$

For eigenstate $|\psi\rangle$, this becomes:

$$\frac{1}{2^{n/2}} \sum_{k=0}^{2^n-1} e^{2\pi i \phi k} |k\rangle \otimes |\psi\rangle.$$

Step 3: Quantum Fourier Transform (QFT)

Applying QFT to the first register transforms the state to:

$$\sum_{k=0}^{2^n-1} c_k |k\rangle,$$

where coefficients c_k are peaked near $k \approx 2^n \phi$.

Step 4: Measurement

Measuring the first register gives an n-bit estimate of ϕ with high probability.

Simple code which implements the QPE

```
import numpy as np
def hadamard(n):
    # Creates an n-qubit Hadamard gate as a matrix.
    H = np.array([[1, 1], [1, -1]]) / np.sqrt(2)
    H_n = H
    for _ in range(n - 1):
        H_n = np.kron(H_n, H)  # Tensor product to expand Hadamard gate
    return H_n

def qft(n):
    # Creates an n-qubit Quantum Fourier Transform (QFT) matrix.
    N = 2**n
    omega = np.exp(2j * np.pi / N)
    qft_matrix = np.array([[omega**(i * j) for j in range(N)] for i in
    return qft_matrix

def inverse_qft(n):
    # Creates an n-qubit inverse Quantum Fourier Transform (QFT†) matri
    return np.conj(qft(n)).T  # Hermitian transpose of QFT

def controlled_unitary(U, control, target, n):
    # Creates an n-qubit controlled unitary matrix
    I = np.eye(2**n)  # Identity matrix
    CU = np.copy(I)
    for i in range(2**n):
        if (i >> control) & 1:  # Check if control qubit is |1⟩
            CU[i, :] = np.kron(np.eye(2**target), U).dot(I[i, :])
    return CU
```

Code which implements the QPE using Qiskit

```
from qiskit import QuantumCircuit, Aer, transpile, assemble, execute
from qiskit.visualization import plot_histogram
import numpy as np

def qpe(unitary, num_counting_qubits):
    """
    Quantum Phase Estimation Algorithm.
    Args:
        unitary (QuantumCircuit): The unitary operator whose phase we e
        num_counting_qubits (int): Number of qubits in the counting reg

    Returns:
        QuantumCircuit: QPE quantum circuit
    """
    n = num_counting_qubits
    qc = QuantumCircuit(n + 1, n)  # n counting qubits + 1 eigenstate q
    # Step 1: Apply Hadamard to counting qubits
    for qubit in range(n):
        qc.h(qubit)
    # Step 2: Apply controlled-U^2^j operations
    for j in range(n):
        power = 2**j
        controlled_U = unitary.control(1).power(power)
        qc.append(controlled_U, [j] + [n])  # Control: j, Target: n
    # Step 3: Apply inverse QFT
    qc.append(qft_dagger(n), range(n))
    # Step 4: Measure counting qubits
    qc.measure(range(n), range(n))
```

Code which implements the QPE using PennyLane

```
import pennylane as qml
import numpy as np
def qft(n):
    # Applies the inverse Quantum Fourier Transform (QFT†) circuit
    for i in range(n):
        qml.Hadamard(wires=i)
        for j in range(i):
            qml.CPhase(-np.pi / (2 ** (i - j)), wires=[j, i])
    for i in range(n // 2):
        qml.SWAP(wires=[i, n - i - 1])

def controlled_unitary(U, control, target):
    # Applies a controlled-unitary operation
    qml.ctrl(U, control=control)(wires=target)


def qpe(phi, num_counting_qubits):
    # Quantum Phase Estimation circuit in PennyLane.
    total_qubits = num_counting_qubits + 1
    dev = qml.device("default.qubit", wires=total_qubits, shots=1000)
    @qml.qnode(dev)
    def circuit():
        # Initialize the counting register in |0> and eigenstate regist
        qml.PauliX(wires=num_counting_qubits)  # Set last qubit to |1>
        # Apply Hadamard to counting qubits
        for qubit in range(num_counting_qubits):
            qml.Hadamard(wires=qubit)
        # Apply controlled-U^2^j operations
```

Applications of QPE

The QPE is a foundational algorithm with several key applications:

1. **Factoring and Cryptography:** Integral to Shor's algorithm for integer factoring.
2. **Quantum Simulations:** Estimating eigenvalues of Hamiltonians in many-body physics.
3. **Amplitude Estimation:** Enhances algorithms like Grover's search.

Challenges and Practical Considerations

1. **Qubit Requirements:** Requires a large number of qubits for high precision.
2. **Gate Depth:** Controlled-U^{2^j} operations increase gate complexity.
3. **Noise Sensitivity:** Errors in QFT or controlled gates impact accuracy.

It is a powerful quantum algorithm for extracting phase information of unitary operators. It forms the foundation for many advanced quantum algorithms and demonstrates the power of quantum Fourier transforms in computational tasks. For many-body simulations it is however not as efficient as the VQE algorithm.

Brief overview of Shor's algorithm

1. Shor's algorithm is a quantum algorithm for integer factorization.
2. It efficiently factors large integers by leveraging quantum period finding.
3. Exponential speedup over the best-known classical algorithms.
4. Breaks RSA encryption, which relies on the difficulty of factoring.

Some history

Shor's algorithm, introduced by Peter Shor in 1994, revolutionized the field of cryptography. It demonstrated that a quantum computer could solve integer factorization and the discrete logarithm problem efficiently, posing a significant threat to classical cryptosystems such as RSA.

Problem Statement

The objective of Shor's algorithm is to factorize a large composite integer N into its prime factors. Given $N = p \times q$, where p and q are unknown distinct primes, the problem of factorization can be reduced to finding an integer a such that $1 < a < N$ and $\gcd(a, N) = 1$.

Reduction to Order-Finding

The key reduction in Shor's algorithm lies in transforming factorization into an order-finding problem:

1. Choose a random integer a such that $1 < a < N$ and $\gcd(a, N) = 1$.
2. Determine the smallest positive integer r such that $a^r \equiv 1 \pmod{N}$.

The integer r is known as the *order* of a modulo N, and it helps in discovering the factors of N.

Continued Fraction Expansion

After obtaining the rational approximation from the quantum algorithm, classical computation using continued fraction expansion aids in deducing the correct order r.

Basics of number theory

1. The problem: Factor an integer N into its prime factors.
2. Reduce factoring to period finding $f(x) = a^x \mod N$, where a is randomly chosen such that $\gcd(a, N) = 1$.
3. The period r satisfies $a^r \equiv 1 \mod N$.
4. Once r is known, factors are given by $\gcd(a^{r/2} \pm 1, N)$.

QFTs again

The Quantum Fourier Transform is crucial to Shor's algorithm. It leverages quantum parallelism and interference to efficiently estimate the periodicity of a function.

$$\text{QFT} : |x\rangle \mapsto \frac{1}{\sqrt{2^n}} \sum_{y=0}^{2^n-1} e^{2\pi i x y / 2^n} |y\rangle$$

The quantum part of Shor's algorithm is designed to find the period r of the modular exponentiation function $f(x) = a^x \bmod N$:

Quantum Period Finding

Quantum Period Finding: Key to Shor's Algorithm

1. Quantum subroutine for finding the period r of $f(x)$.
2. Similar to Quantum Phase Estimation (QPE).
3. Uses two quantum registers:
   - First register: Superposition of all possible inputs x.
   - Second register: Computes $f(x) = a^x \mod N$.

Applying Quantum Fourier Transform (QFT)

1. The QFT is applied to the first register after the controlled unitary operations.
2. Peaks in the measurement correspond to multiples of $1/r$.
3. Post-processing classically determines r by continued fractions.

Classical Post-Processing

1. Once r is found, factors of N are computed by $\gcd(a^{r/2} \pm 1, N)$.
2. If r is odd or $a^{r/2} \equiv -1 \mod N$, try a different a.
3. With high probability, this yields a non-trivial factor of N.

Complexity and Practical Considerations

1. Quantum Complexity: Polynomial in $\log N$.
2. Classical Best Known: Sub-exponential (Number Field Sieve).
3. Quantum Advantage: Exponential speedup over classical algorithms.
4. Practical Challenges: Quantum error correction, large qubit counts.

Error Analysis and Success Probability

The probability that a randomly chosen a leads to successful factoring is generally greater than 50%. The quantum component requires carefully managed resources for high precision.
Error correction is crucial, as any errors in operations or QFT can significantly impact the accuracy of period estimation and the eventual factorization.

Summarizing Shor's algorithm

Shor's algorithm is a central quantum algorithm demonstrating exponential speedup over classical counterparts. It brought both excitement for potential quantum advancements and a reconsideration of current cryptographic standards.

1. Shor's algorithm is a groundbreaking quantum algorithm for factoring.
2. Combines quantum period finding with classical number theory.
3. Highlights the potential of quantum computing to break RSA.